Что такое подстрока?

Пусть P это паттерн, а T - текст. Задача о точном совпадении заключается в отыскании всех вхождение паттерна P в текст T.

Строка S - упорядоченный список символов. Для любой строки S обозначим **подстроку**, как $S[i;j]$, которая начинается в i-ой и заканчивается в j-ой позициях. *Префиксом* строки S называется подстрока $S[0;i]$, а *суффиксом* - $S[i;|S|-1]$.

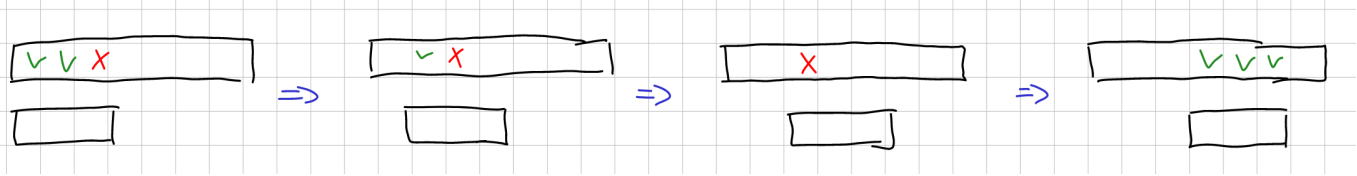
Собственные префикс, суффикс и подстрока строка S - не пустные и не совпадающеие с иходной строки

Наивный алгоритм

Алгоритм:

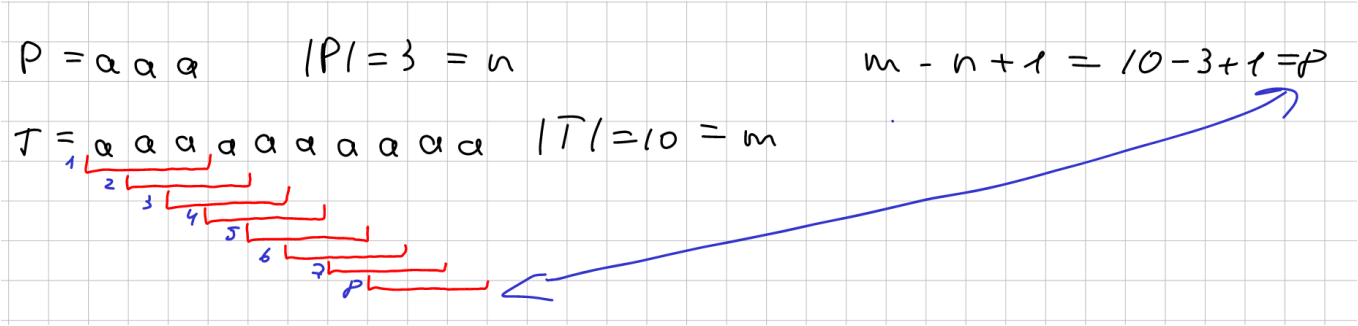
- Левый конец P ставится на одну позицию с левым концом T.
- Начинаем посимвольно сравнивать элементы слева направо, пока либо не встретится несовпадение, либо не закончится P.
  - Если P закончился, то найдено вхождение P в T.
- При любом исходе P сдвигается ровно на одну позицию вправо и алгоритм сравнения повторяется с левого конца P.
- Сравнения заканчиваются, когда правый конец P выйдет за правый конец T.

Пусть $|P|=n$, а $|T|=m$, тогда **сложность алгоритма** будет
$O(n\cdot(m-n+1))=O(nm+n^2+n)=O(nm)$



Сколько сравнений потребуется, если символы в P и T одинаковы.

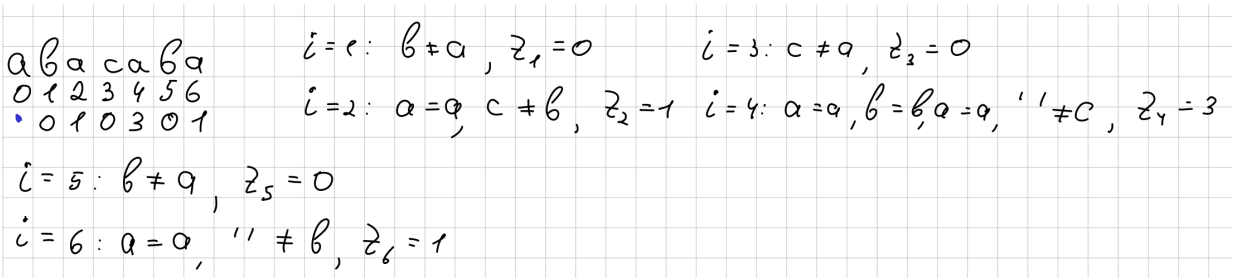
Если P и T состоят из одного и того же повторяющегося символа, то P входит в текст в каждой из $m-n+1$ позиций и метод выполняет ровно $n(m-n+1)$ сравнений.



Z-функция

Для строки S и позиции $i > 0$ определим Z_i как длину наибольшей подстроки S, которая начинается в i и совпадает с префиксом исходной строки S.
Наивный алгоритм подсчёта Z-функции:

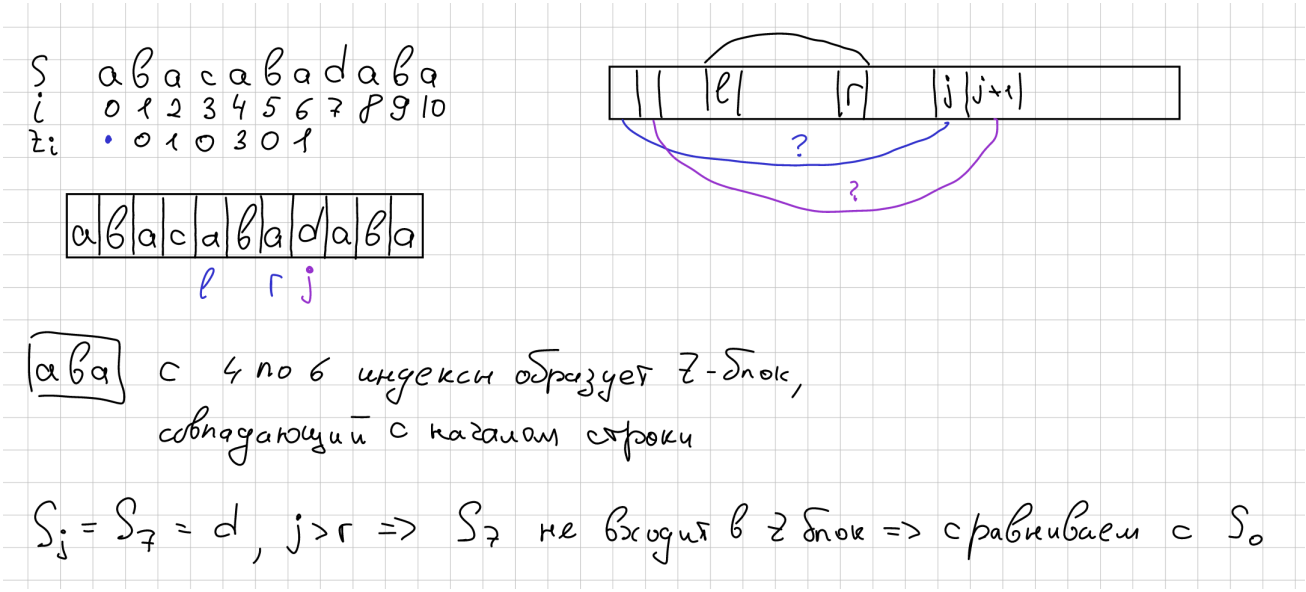
1. Для 0 индекса сравнивать не с чем, поэтому он остаётся пустым - заглушкой.
2. Для всех индексов больше нуля начинаем считать Z-функию следующим образом:
    1. Пусть находимся в позиции i. Тогда необходимо сравнить S_0 и S_i.
    2. В случае совпадения продолжаем сравнение для S_1 и S_{i+1}.
    3. Как только случилось несовпадение в Z_i записываем количество совпавших символов.
    4. Сдвигаем i на одну позицию и начинаем сравнение заново с п.2.1.
    Сложность такого алгоритма в худшем случае будет $O(n^2)$, где $n = |S|$.
    Пример:



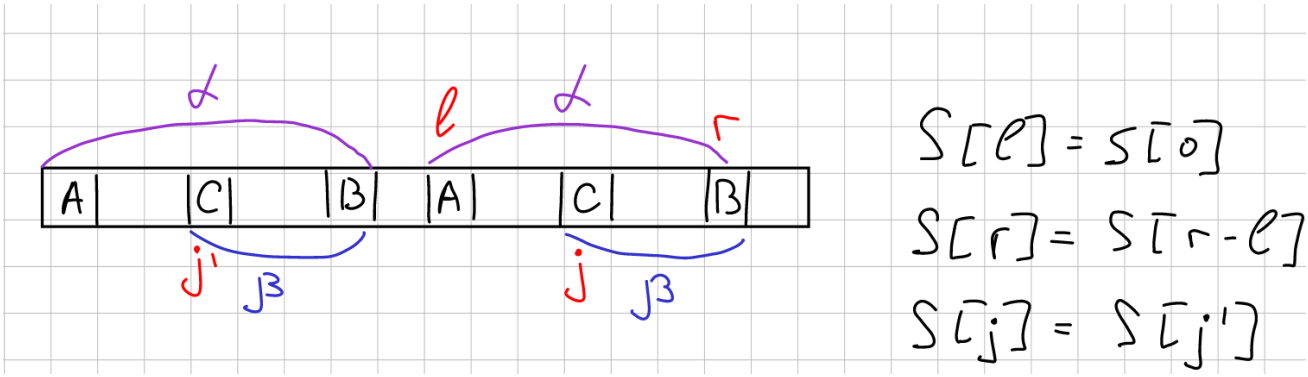
Ускорение для линейной сложности

Введём переменные l и r, как края, обозначающие промежуток, для которого уже была посчитана Z-функция, причём $r = l + Z_i - 1$, $l = i$. Тогда для j индекса большего чем i при подсчёте Z-функции возможны 2 ситуации:

1. $j > r$. В таком случае мы просто начинаем сравнение с началом строки для $S_0 \ ? \ S_j, S_1 \ ? \ S_{j+1}, \dots$

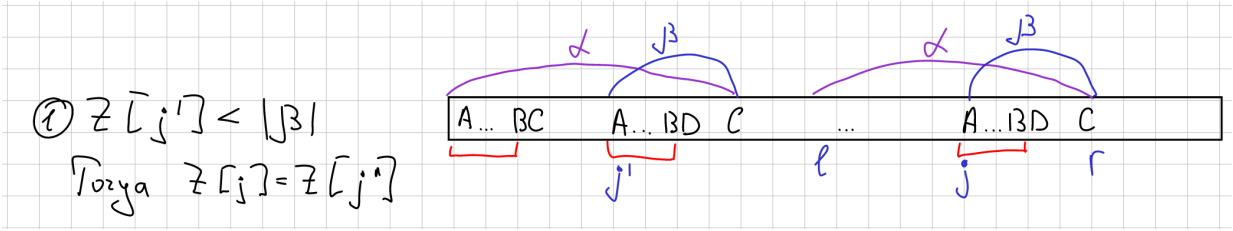


2. $l < j < r$. В таком случае S_j находится в Z-блоке α, для которого уже было вычислено количество совпадений.



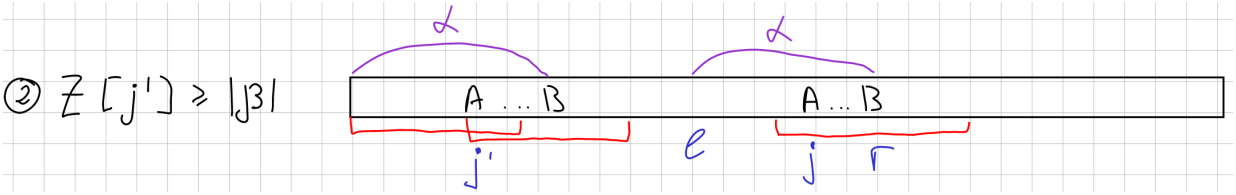
Причём $S_0 = S_l, \dots S_{j'} = S_j, S_{r-l} = S_r$, $j' = j - l$. И тут возможные ещё 2 ситуации:

1. $Z_{j'} < |\beta|$, $|\beta| = r - j + 1$, а сам β - остаток α совпадающий с j индекса с j' индексом начала строки. И если Z-функция для j' меньше длины этого отрезка, то возможный Z-блок поглощается блоком, заканчивающимся позже. Поэтому $Z_j = Z_{j'}$.



2. $Z_{j'} \geq |\beta|$. Тогда в S_j начинается новый Z-блоком, правая граница которого лежит за правой границей текущего рассматриваемого блока. Элементы между j и r индексами включительно можно не сравнивать, они точно совпадают, а дальше начинается сравнение S_{r+1} с $S_{|\alpha|+1}$, количество этих совпадений примем за M. В

итоге $Z_j = Z_{j'} + M, l = j, r = j + Z_j - 1$.



Тогда алгоритм за линию в псевдокоде имеет вид:

```
s = строка
n = |s|
z = массив длины n, заполненный 0
left, right = 0, 0
for i от 0 до n-1
     если i < right
          z[i] = min(right - i + 1, z[i - left])

     пока (i + z[i] < n) И (s[z[i]] == s[i + z[i]])
          z[i]++

     если i + z[i] - 1 > right
          left = i
          right = i + z[i] - 1
```

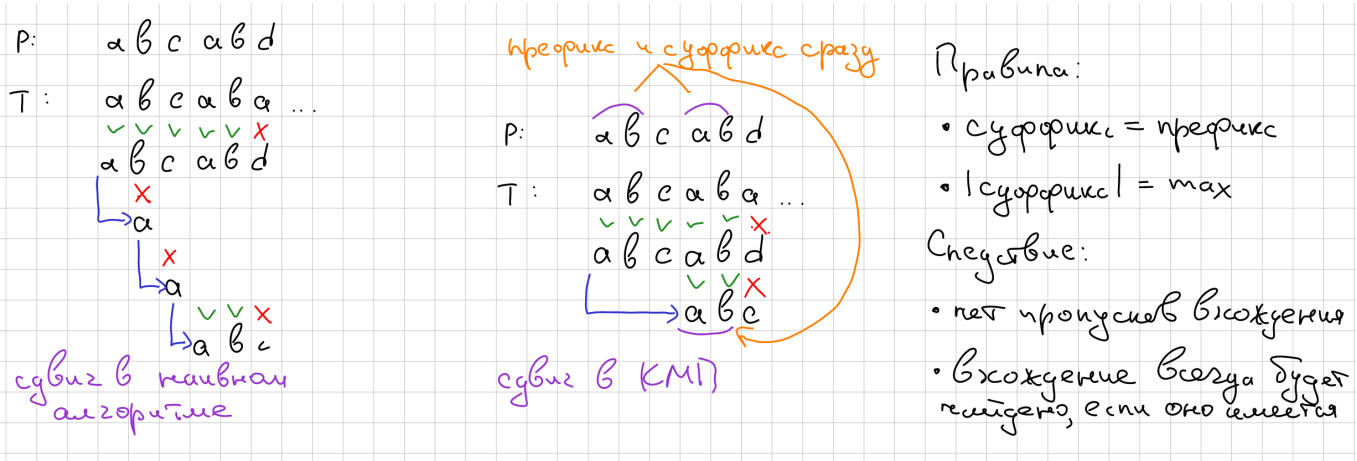
Поиск подстроки в строке с помощью Z-функции

Алгоритм:

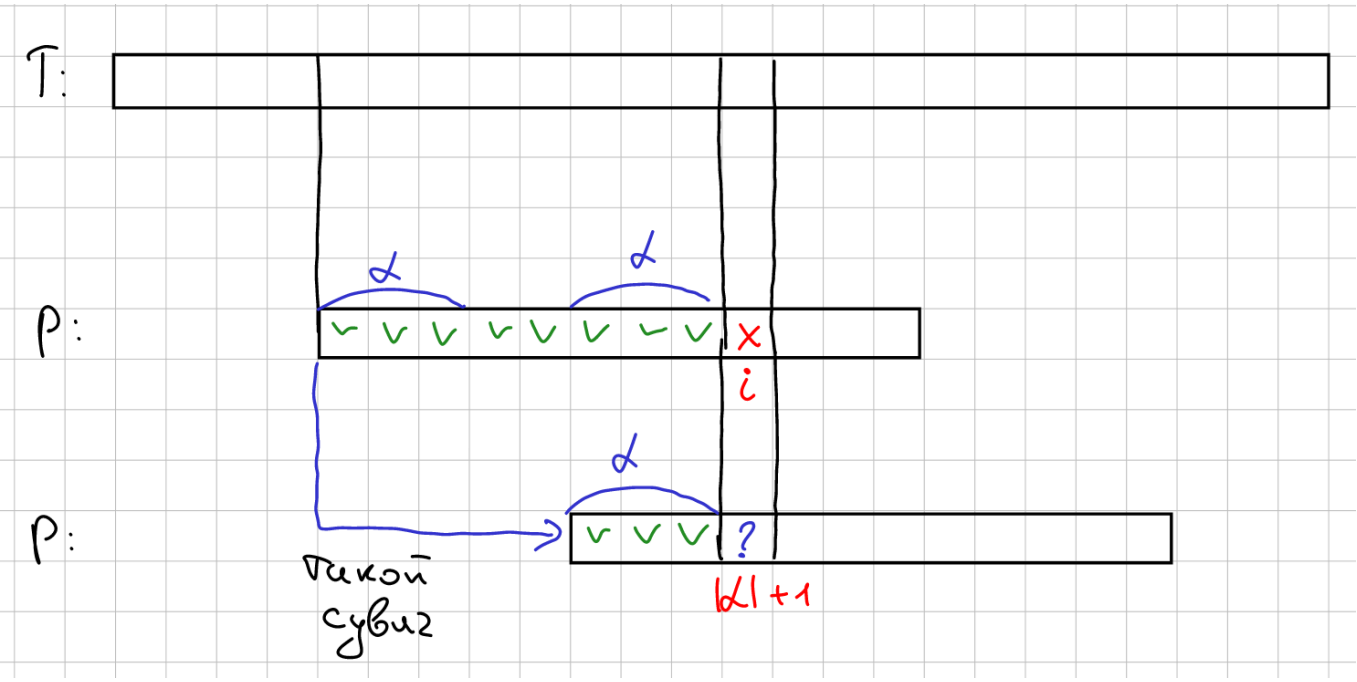
1. Объединяем паттерн P и текст T в одну строку с разделителем в виде уникального терминального символа
2. Считаем Z-фукнцию для получившейся строки
3. Проходим по массиву Z-функции, если встречаем число, равное длине паттерна P, то это вхождение. После полного прохода по Z-массиву будут найдены все вхождения паттерна P в текст T.
   Сложность: $O(|S|) = O(n + m)$.

КМП

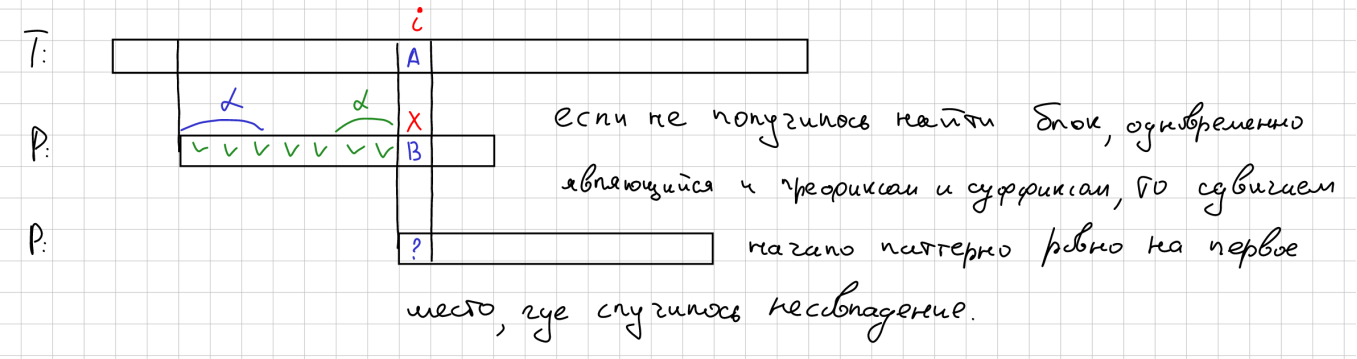
Правило сдвига



В общем виде:



Если не получилось найти блок, одновременно являющийся и префиксом, и суффиксом, то сдвигаем начало паттерна ровно на первое место, где случилось несовпадение



Препроцессинг

SP-функция

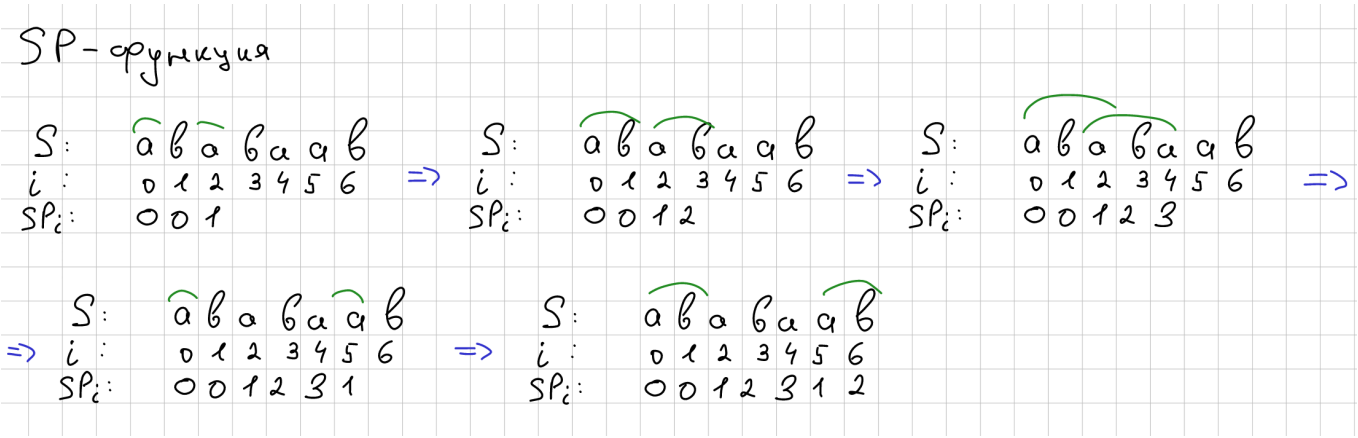
Определение: $sp[i]$ — это длина максимального собственного префикса строки $S[0 \dots i]$, который одновременно является её суффиксом.

Алгоритм (вычисляем массив sp для строки S длины n):

1. Базовый случай: $sp[0] = 0$ (у строки из одного символа нет собственного префикса).
2. Заводим переменную `j = 0`. Она будет хранить длину текущего совпавшего префикса (и одновременно индекс следующего символа, с которым мы будем сравнивать $S[i]$).
3. Проходим циклом `for` по i от 1 до $n - 1$:
   - **Пока** `j > 0` и $S[i] \neq S[j]$, мы откатываем `j` назад: `j = sp[j - 1]`. (Пытаемся найти префикс поменьше).
   - **Если** $S[i] == S[j]$, то мы можем удлинить префикс: `j++`.
   - Записываем результат: `sp[i] = j`.

Сложность: $O(n)$ по времени и памяти.

Пример вычисления SP-функции



i	**Символ** $S[i]$	**Текущее** j	**Сравнение** $S[i]$ **и** $S[j]$	**Действие / Откат**	**Результат** $sp[i]$
0	**a**	0	-	Базовый случай	**0**
1	**b**	0	'b' $\neq$ 'a' ($S[0]$)	j уже 0, совпадений нет	**0**
2	**a**	0	'a' $==$ 'a' ($S[0]$)	Совпало! $j \to 1$	**1**
3	**b**	1	'b' $==$ 'b' ($S[1]$)	Совпало! $j \to 2$	**2**
4	**a**	2	'a' $==$ 'a' ($S[2]$)	Совпало! $j \to 3$	**3**

i	**Символ** $S[i]$	**Текущее** j	**Сравнение** $S[i]$ **и** $S[j]$	**Действие / Откат**	**Результат** $sp[i]$
5	**a**	3	'a' $\neq$ 'b' ($S[3]$)	Не совпало! Откат: $j = sp[2] = 1$. Сравниваем 'a' с $S[1]$ ('b') - нет. Откат: $j = sp[0] = 0$. Сравниваем 'a' с $S[0]$ ('a') - да! $j \to 1$	**1**
6	**b**	1	'b' $==$ 'b' ($S[1]$)	Совпало! $j \to 2$	**2**

Итог: $sp = [0, 0, 1, 2, 3, 1, 2]$

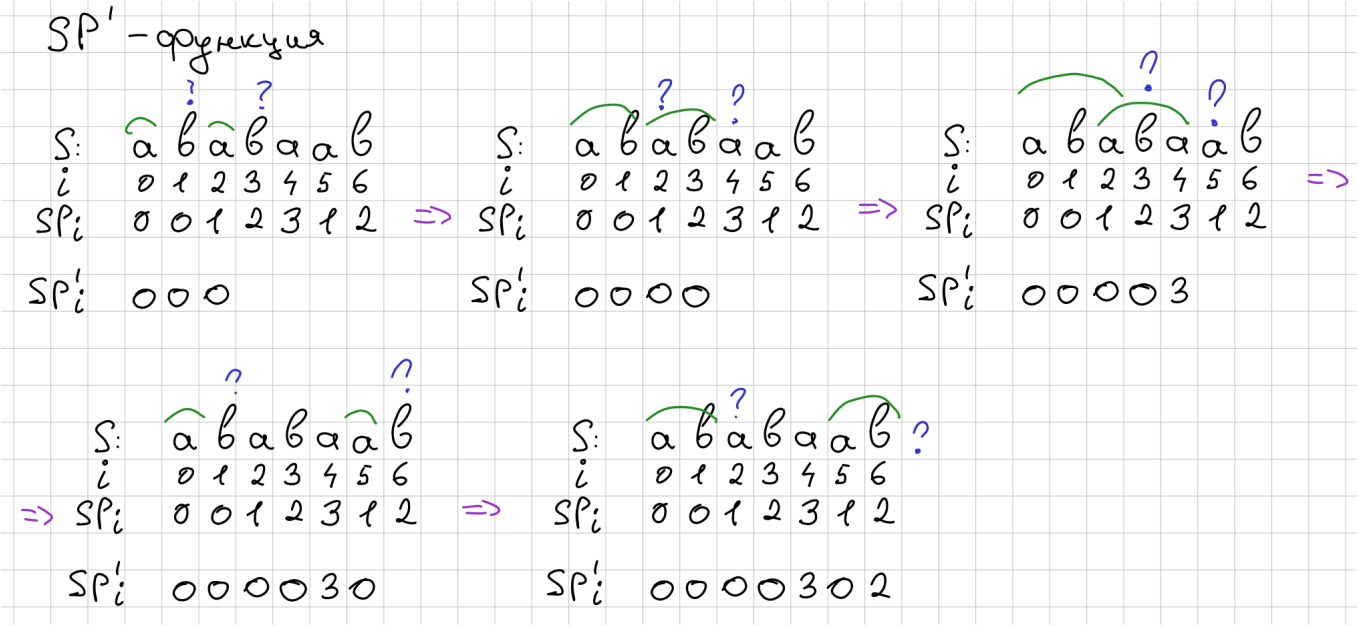
SP'-функция

Зачем нужна: При обычном КМП, если произойдет несовпадение текста с символом $S[i+1]$, мы сделаем сдвиг и будем сравнивать текст с $S[sp[i]]$. Но если $S[sp[i]] == S[i+1]$, то новое сравнение **гарантированно приведет к ошибке**, и мы потратим шаг впустую. **Сильная SP-функция ($sp'[i]$)** — это длина такого максимального префикса-суффикса, за которым следует символ, **отличный** от $S[i+1]$. То есть: $S[sp'[i]] \neq S[i+1]$.

Алгоритм (вычисляется за $O(n)$ на основе обычного sp):

1. Сначала вычисляем обычный массив sp.
2. Создаем массив sp'. (Для последнего элемента $sp'[n-1] = sp[n-1]$, так как за ним нет символа).
3. Проходим по i от 0 до $n-2$:
   - Если $sp[i] == 0$, то $sp'[i] = 0$.
   - Иначе смотрим на символы, следующие за префиксом и суффиксом:
     - Если $S[sp[i]] \neq S[i+1]$, то нас всё устраивает: $sp'[i] = sp[i]$.
     - Если $S[sp[i]] == S[i+1]$, сдвиг по $sp[i]$ бесполезен. Мы берем "сильное" значение от того префикса, в который бы перешли: $sp'[i] = sp'[sp[i]-1]$.

Пример вычисления SP'-функции



i	$S[i]$	$sp[i]$	**След. символ за префиксом ($S[sp[i]]$)**	**След. символ в паттерне ($S[i+1]$)**	**Сравнение**	**Результат $sp'[i]$**
0	**a**	0	$S[0] =$ **a**	$S[1] =$ **b**	**a** $\neq$ **b**	**0**
1	**b**	0	$S[0] =$ **a**	$S[2] =$ **a**	**a** $==$ **a**	**0**
2	**a**	1	$S[1] =$ **b**	$S[3] =$ **b**	**b** $==$ **b**	**0** (Берем $sp'[1-1] = sp'[0]$)
3	**b**	2	$S[2] =$ **a**	$S[4] =$ **a**	**a** $==$ **a**	**0** (Берем $sp'[2-1] = sp'[1]$)
4	**a**	3	$S[3] =$ **b**	$S[5] =$ **a**	**b** $\neq$ **a**	**3** (Хороший сдвиг, символы разные)
5	**a**	1	$S[1] =$ **b**	$S[6] =$ **b**	**b** $==$ **b**	**0** (Берем $sp'[1-1]$)
6	**b**	2	—	—	Последний символ	**2** (Обычно оставляем sp)

Итог: $sp' = [0, 0, 0, 0, 3, 0, 2]$

Алгоритм (на основе Z-функции):

1. Посчитать Z-функию для строки
2. Начать обход по Z-массиву с конца.
3. Пользуясь формулой $i = j + Z_j - 1$ вычислить сильную префикс функцию $SP'_i = Z_j$.

Почему суммарное число сравнений при вычислении префикс-функции линейно от длины строки?

Хотя внутри цикла `for` (от 1 до $n-1$) есть цикл `while`, алгоритм работает за $O(n)$. Для доказательства используется метод амортизационного анализа.

Рассмотрим переменную `j` (длину текущего префикса-суффикса):

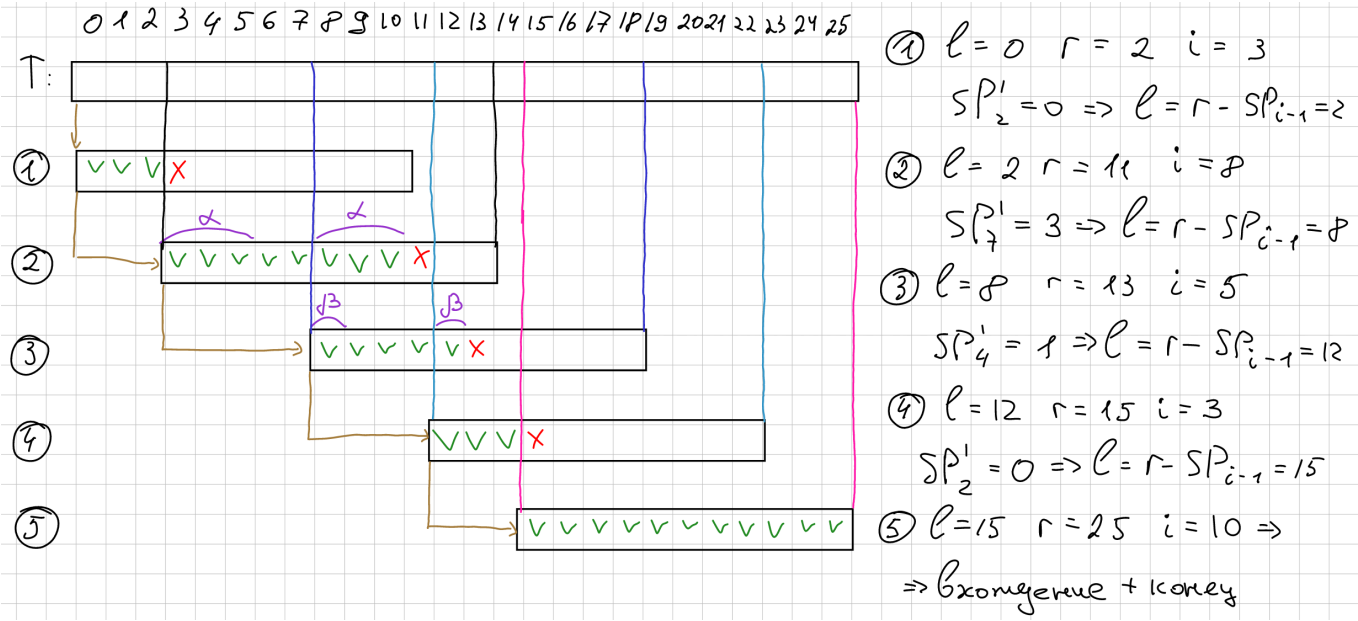
1. Изначально `j = 0`.
2. В каждой итерации цикла `for` значение `j` может увеличиться **максимум на 1** (в строке `j++`). Значит, за весь алгоритм `j` увеличится не более чем $n-1$ раз.
3. В цикле `while` при несовпадении мы делаем `j = sp[j - 1]`. Так как $sp[k] < k+1$, значение `j` строго **уменьшается** как минимум на 1.
4. Поскольку `j` не может стать отрицательным, суммарное количество уменьшений `j` (итераций цикла `while`) не может превышать суммарного количества увеличений `j`.
5. Следовательно, общее число итераций цикла `while` за всё время работы алгоритма ограничено числом n. Общее время работы строго пропорционально длине строки: $O(n)$.

Алгоритм КМП с применением SP'_i

Алгоритм:

1. Для паттерна P посчитать Z-фукнцию(или SP-функцию).
2. Для паттерна P посчитать SP', используя метод из п.1.
3. Установить указатель $l = 0$, который показывает позицию, откуда начинается сравнение паттерна с текстом. Установить указатель $r = 0$, который показывают позицию сравниваемого в данной итерации символа строки с паттерном.
4. Начать посимвольно сравнивать паттерн с текстом.
5. Если на индексе i паттерна P и индексе r текста T произошло несовпадение символов, то необходимо посмотреть SP_{i-1}. Далее сделать перенос левой границы приложения паттерна в тексте $l = r - SP_{i-1}$. Начать сравнивать паттерном с новом местом в тексте начиная с начала паттерна ($i = 0$).
6. Если **ВЕСЬ** прошли весь паттерн и не случилось несовдание, то нашли вхождение паттерна P в текст T. Указатель $l = r$.
7. Продолжать алгоритм, пока $r \leq |T|$.

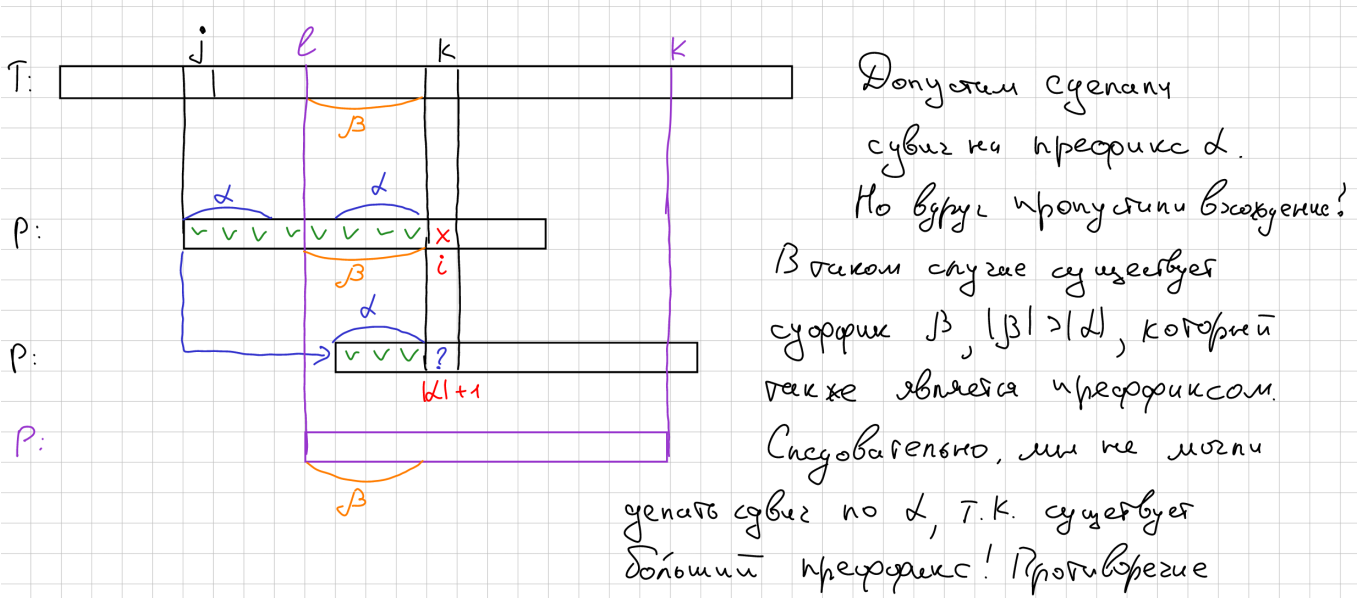
Пример в общем виде:



Сложность: $O(n-m)$ сдвигов и $O(n)$ сравнений, что в сумме даёт $O(n-m+n) = O(2n-m) = O(2n) = O(n)$, n - длина текста, m - длина паттерна.

Доказательство корректности алгоритма

Доказательство, что нет пропусков вхождений:



Допустим сделали сдвиг на префикс α. Но оказалось, что было прощуено вхождение. Тогда существует суффикс $\beta : |\beta| > |\alpha|$, который также является префиксом. *Следовательно*, нельзя было сдвигать по α, т.к. существует префикс большей длины. *Противоречие!*

Максимальное количество сравнений в КМП? Доказательство.

Теорема: В КМП количество сравнений не превышает $2m$.
Доказательство: Алгоритм КМП делится на фазы сравнения и фазы сдвига. Сравнения происходят после каждого сдвига и начинаются с последнего сравниваемого символа в T. Получается в каждой фазе сравнения будет не более m сравнений. А суммарное количество будет не более $m + s$ сравнений, где s - количество сдвигов. Очевидно, что в наихудшем случае будет $s = m$ сдигов, *следовательно*, общее количество сравнений никогда не превысит $2m$.

КМП реального времени

> Из заметок

Вместо SP'_i ищем $SP'_{i,A}$, где A - какой-то символ в тексте который не совпал с паттерном; мы ищем такой префикс совпадает с суффиксом, за которым идет буква A. Если далее идет не A, то $SP'_{i,A} = 0$.

Бойер-Мур

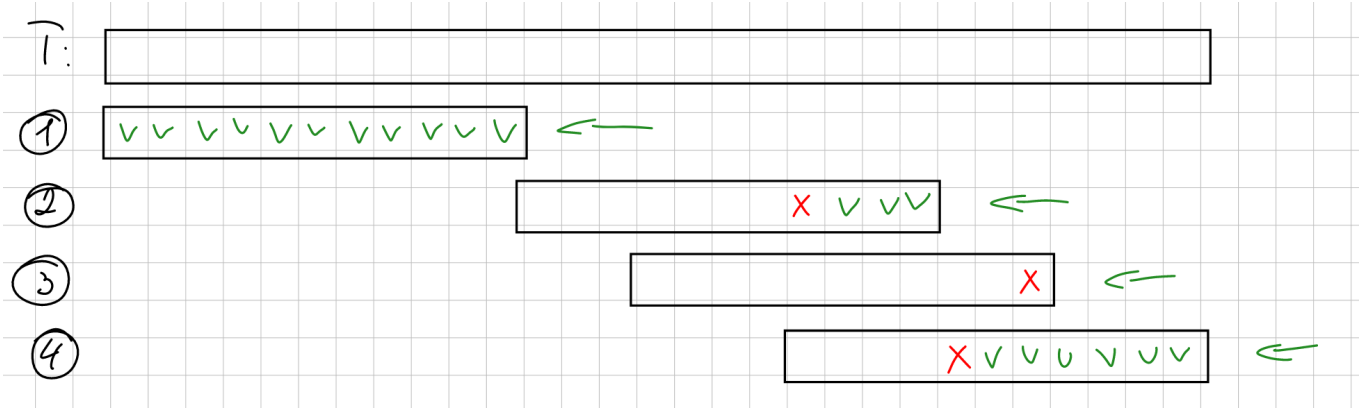
Основная идея

Пусть дан текст T длины n и паттерн P длины m.
Левый конец паттерна устанавливается на уровне левого конца текста. Начинается посимвольное сравнение с правого конца символа, т.е. сравниваются T_m ? P_m, T_{m-1} ? P_{m-1}, ... T_0 ? P_0.

- Если все сравнения дают *истину*, то это полное вхождение паттерна P в текст T. После чего левый конец паттерна устанавливается на позицию $m + 1$ текста и начинается посимвольное сравнение справа налево заново, пока не будет обработан весь текст.
- Если хотя бы одно сравнение символов даёт *ложь*, то левая позиция паттерна сдвигается с использованием правила плохого символа и/или правила хорошего суффикса. После чего начинается новое сравнение паттерна с текстом справа налево, пока не будет обработан весь текст.

Недостатком такого алгоритма является то, что один символ текста может сравниваться с паттерном несколько раз, что видно в итерация 3 и 4 примера ниже, когда в итерации 3 сравнение символа дало несовпадение, а в 4 итерации он проверялся заново и дал совпадение. Однако сложность алгоритма остаётся линейной, причём с констаной меньше чем в КМП.

Пример в общем виде:



Препроцессинг

Препроцессинг подразуемевает собой обработку паттерна для быстрого поиска в тексте. Существует два типа препроцессинга в алгоритме Бойера-Мура. Их можно как использовать по отдельности, так и комбинировать. При этом **минимальный шаг** для смещения паттерна остаётся равен одному символу.

Правило плохого символа (ППС)

> Зайцев называет это правилом плохого литера.

Как делать препроцессинг паттерна:

1. Создается хеш-мапа.
2. При обходе паттерна слева направо в хеш мапу записывается пара <ключ, значение>, где *ключ* - это char, а *значение* - это его индекс. Если ключ в мапе уже встречался, то индекс перезаписывается, т.к. необходимо помнить самую правую позицию.

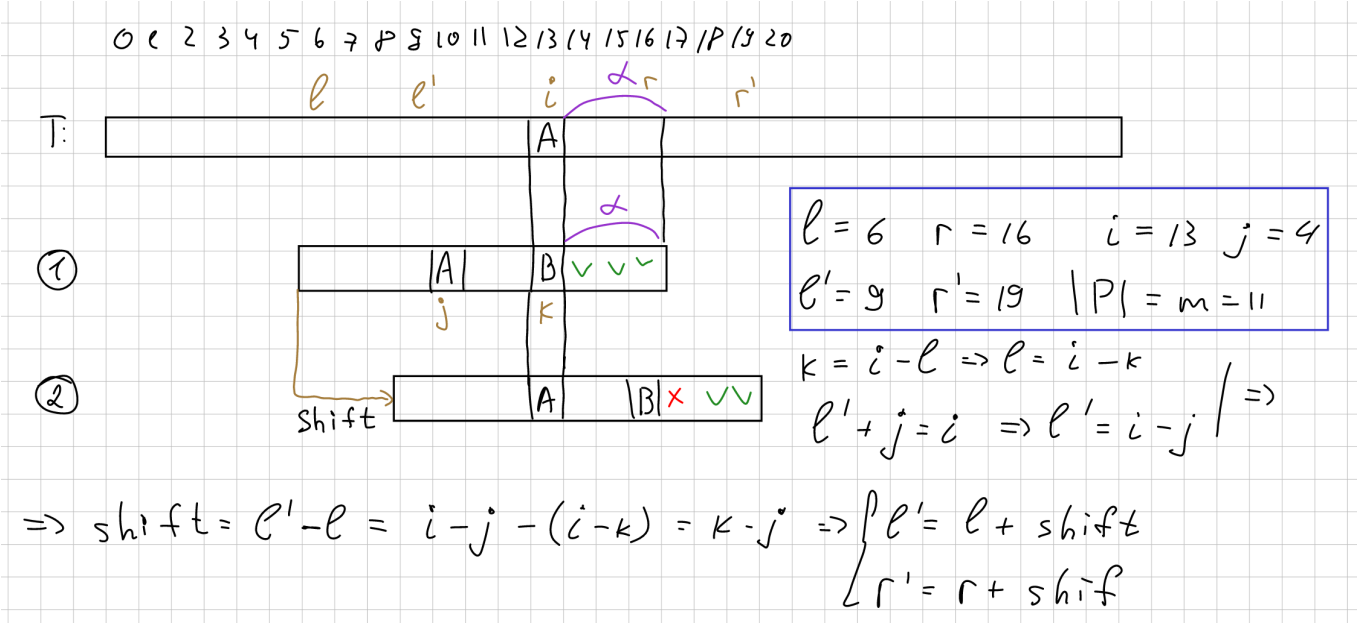
Как использовать в БМ:
Будем считать, что при приложении паттерна к тексту нашлось совпадение $\alpha : |\alpha| = r - i$, причём $T_i \neq P_k$, $k = i - l$.

- Если мы знаем, что $T_i = P_j$, где $j \leq m - 1$. В таком случае следует перенести паттерн так, чтобы P_j находилось под T_i, т.е. $l' + j = i \Rightarrow l' = i - j$. Само перемещение левой границы вычисляется, как $shift = l' - l$, где $l' = i - j$ и $l = i - k$. Получаем $shift = (i - j) - (i - k) = k - j$. Правая граница сдвигается на одно и то же расстояние, из чего образуется система уравнений:

$$\begin{cases} l' = l + shift = l + k - j \\ r' = r + shift = r + k - j \end{cases}$$

- Если же не существует $T_i = P_j$, то

$$\begin{cases} l' = l + k + 1 \\ r' = r + k + 1 \end{cases}$$



Но возможна ситуация, что $l + j > i$. Тогда это свидельствует о том, что к позиции i паттерн позицией j уже был проверен. В таком случае обращаются за правилом **сильного плохого символа**.

Правило сильного плохого символа

Препроцессинг правила сильного плохого символа:

- Создаётся хеш-мапа.
- При обходе паттерна слева направо обращаемся по ключу в хеш-мапу, ключём является char.
  - Если такого ключа нет, то создаем пару <ключ, значение>, где ключ это char, а значением является отсортированный массив, куда будут складываться индексы, где встречается этот char
  - Если ключ есть, то в конец массива его индексов добавляется текущий.

Применение в БМ:

- Если мы нашли какой-то совпадающий блок, за которым есть несовпадение, то обращаемся к хеш-мапе по ключу, равному char в тексте. Выбираем $j : j < k$.
  - Если такой j существует, то делаем перенос:

$$\begin{cases} l' = l + shift = l + k - j \\ r' = r + shift = r + k - j \end{cases}$$

  - Если такого j не существует, то делаем перенос:

$$\begin{cases} l' = l + k + 1 \\ r' = r + k + 1 \end{cases}$$

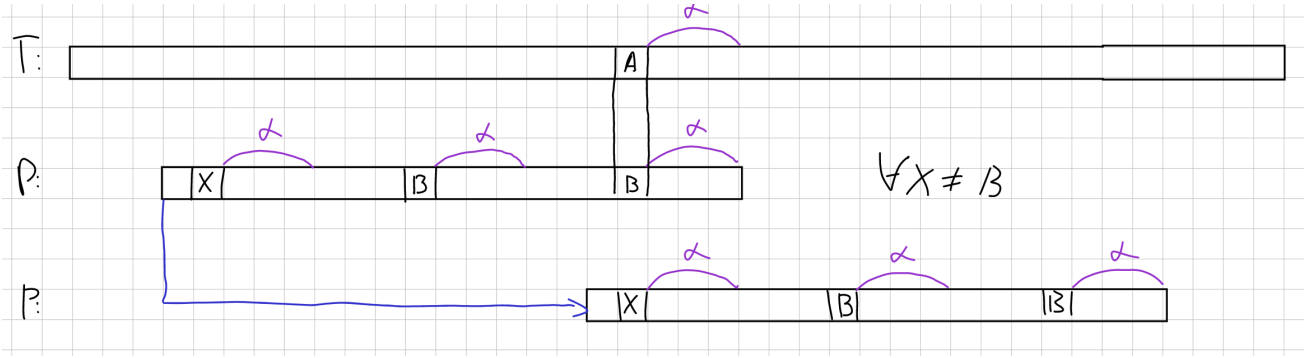
Правило хорошего суффикса (ПХС)

Алгоритм препроцессинга:

1. Взять паттерн P и реверсивно его записать в P'
2. Для P' посчитать Z-фукнцию
3. Введём N-функцию и массив N, размер которого равен длине паттерна P ($|P| = m$) и изначально он заполнен нулями
4. Пользуясь формулой $i = m - j + 1$ заполнить массив N $\forall Z_j \neq 0 : N_i = Z_j$.

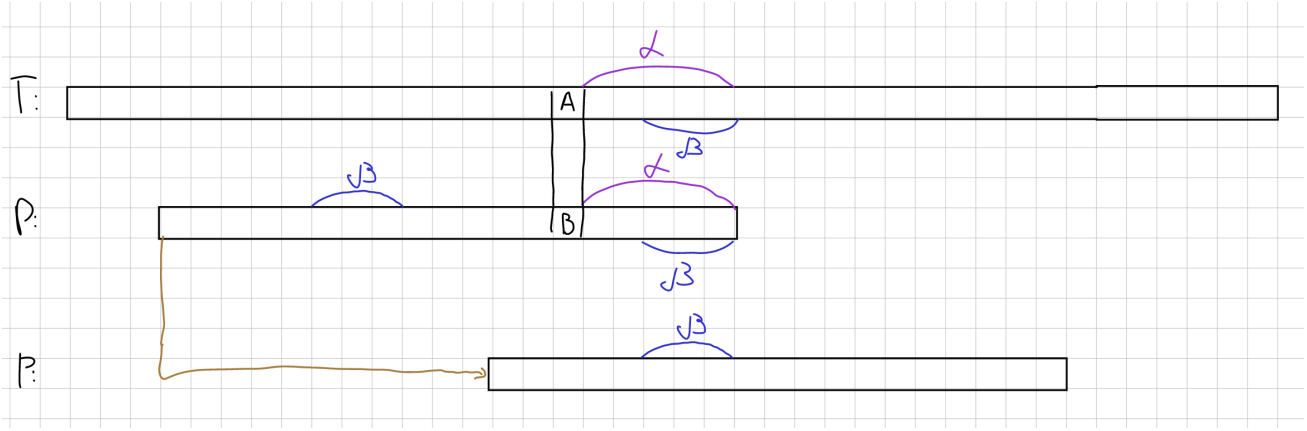
Как применять в БМ:
Если при прикладывании паттерна P к тексту T произошло совпадение подстроки α, причём следующие символы не дали положительного результата сравнения и в паттерне встречается ещё хотя бы одно вхождение подстроки α, то выбирается самое правое вхождение этой подстроки в паттерн так, чтобы символ перед ней не совпадал с тем, что в данный момент разошёлся с текстом.

- Если такое вхождение было найдено, то необходимо передвинуть паттерн так, чтобы это правая подстрока α совпала с тем самым местом в тексте, где она встретилась.



- Если такое вхождение не было найдено, то попытаться внутри подстроки α найти суффикс β, который также может ещё хотя бы один раз встречаться в паттерне и

провести перенос для него аналогично предыдущему пункту.



Совместное применение ППС и ПХС

Для наибольше эффективности шаг перемещение рекомендуется выбирать как max(ППС, ПХС, 1)

Сложность:
Для любой из рассмотренных реализаций препроцессинг будет занимать линейное время - $O(m)$. Аналогично поиск подстроки в строке будет занимать линейное время $O(n)$.
Итоговая сложность: $O(n+m)$, n - длина текста, m - длина паттерна.

Я очень не уверен во всём БМ!

Апостолико Джанкарло

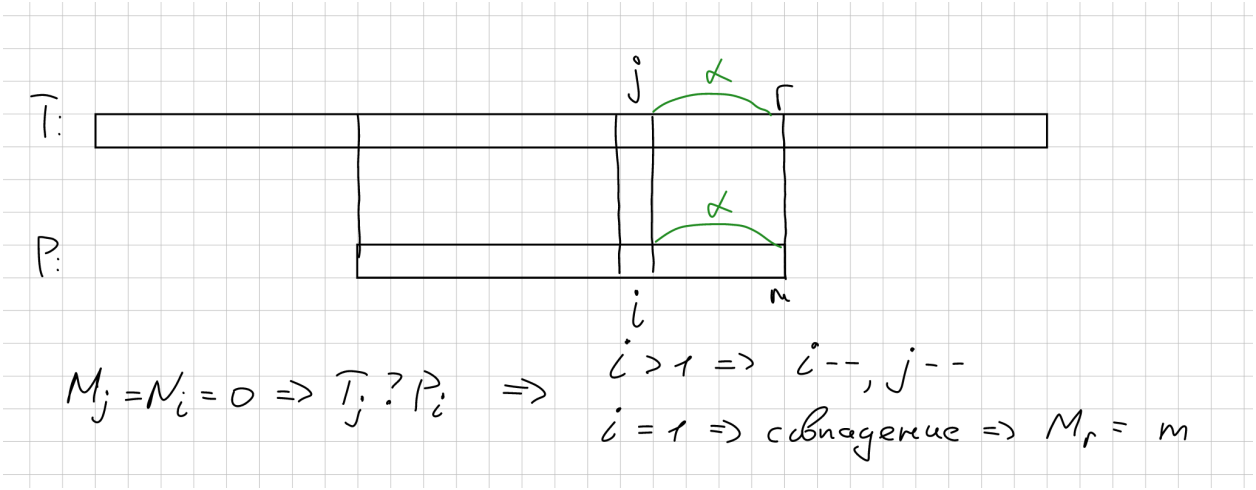
Это штука является просто адаптацией БМ. В чём проблема БМ? В том, что при перестановке паттерна относительно текста у нас уже есть очевидно проверенная часть, которую при новой подстановке всё равно приходится посимвольно проверять. Эти двое чуваков предложили идею, что давай-те покроем текст блоками, которые уже проверены, и просто будем их перескакивать, после переноса паттерна.

Весь алгоритм устроен на адаптации ПХС.

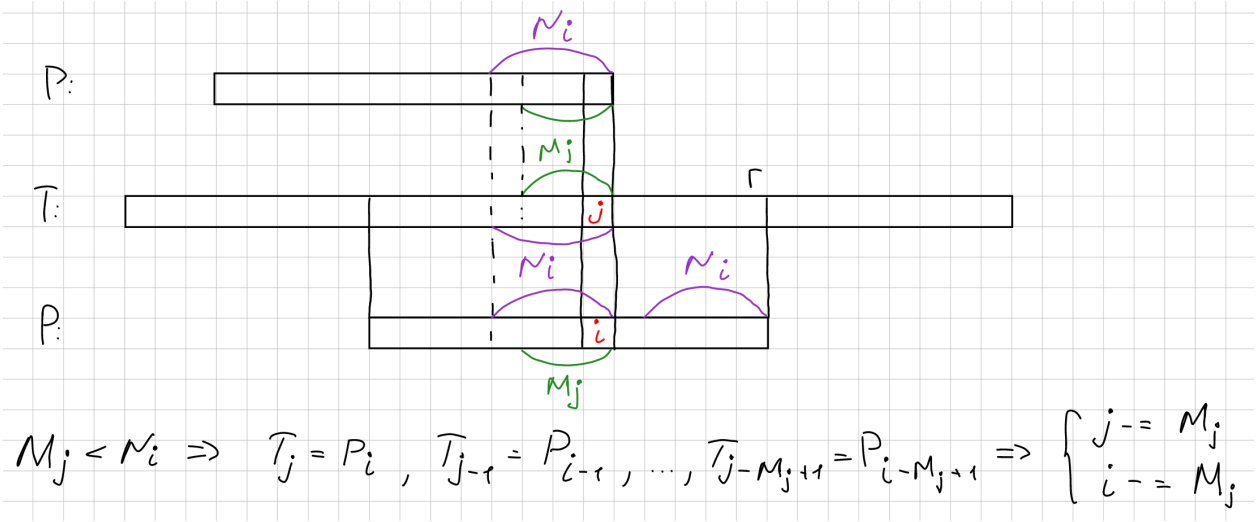
Алгоритм:

1. Посчитать N-фукнцию.
2. Задать новый массив M того же размера, что и текст. Базово задать все значения как 0.
3. Когда начинаем сравнивать текст с позиции j с паттерном в позиции i, то смотрим значение в M_j. Если $M_j = 0$, то из позиции j в тексте не существует подстроки, которая раньше встречалась в паттерне. Если $M_j \neq 0$, то возможны следующие случаи:

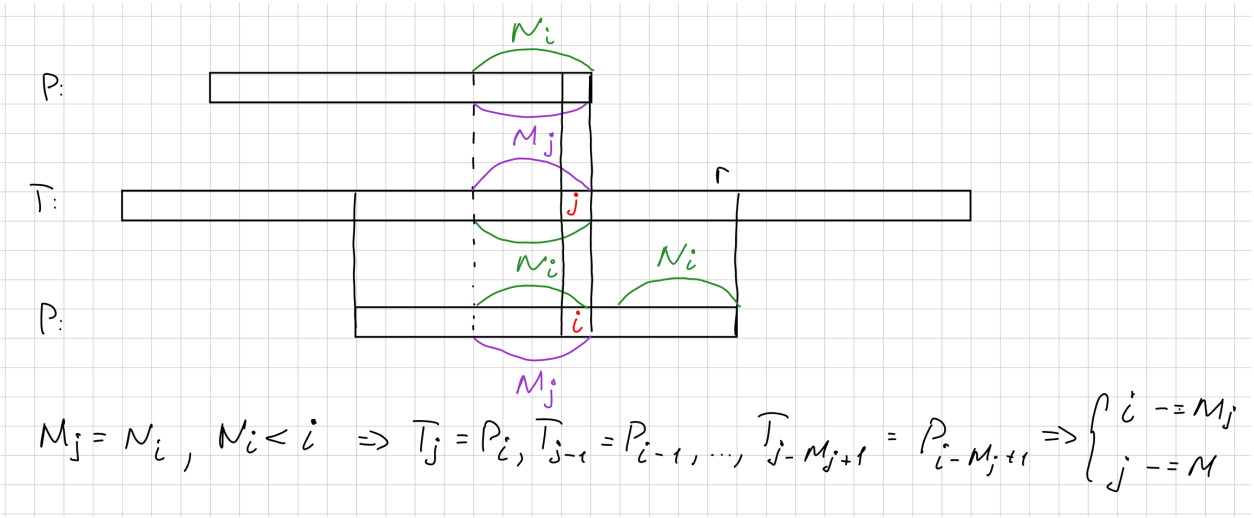
- $M_j = N_i = 0$ (если базовое значение M_j было отличное от 0 и встретилось оно, то попадает в этот же случай). Если $T_j = P_i$ и $i > 0$, то уменьшаем i и j на единицу; если $i = 0$, то найдено вхождение, в M_r записывается длина паттерна (r - позиция, куда приложен правый конец паттерна) - $M_r = |P|$. Если же $T_j \neq P_i$, то $M_r = r - j$.



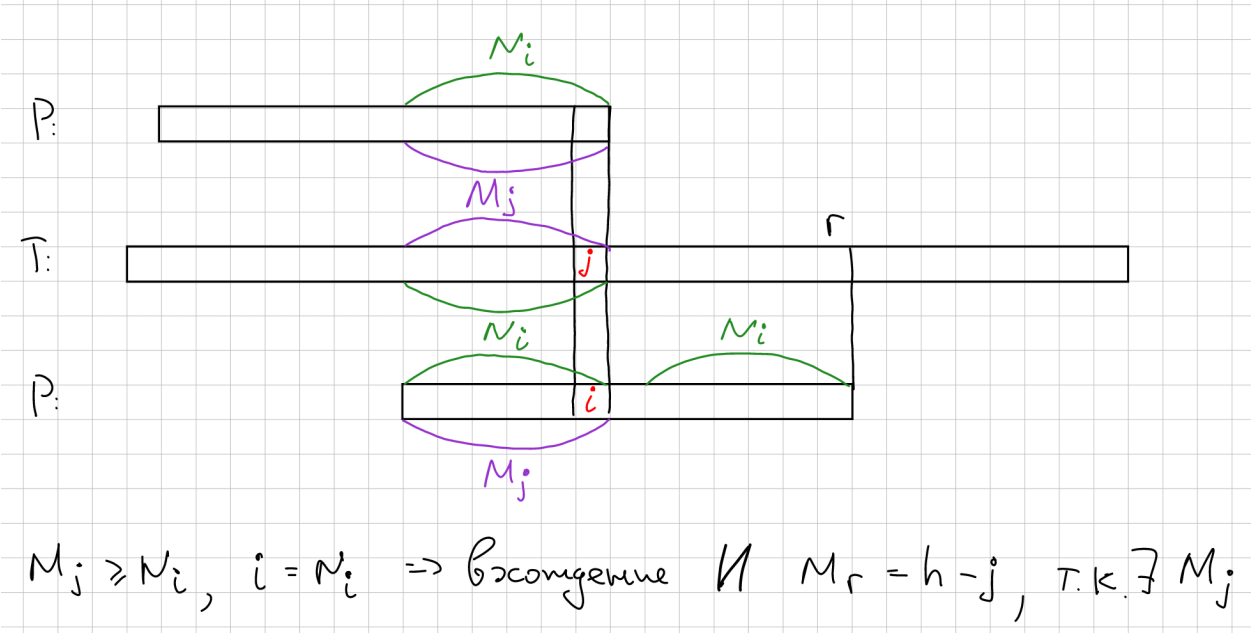
- $M_j < N_i$. В данном случае, если к j позиции текста поставить правый конец паттерна, то с текстом совпадёт M_j последних символов паттерна, при этом существует подстрока длиной N_i, которая также совпадает, начиная с этой позиции (она ещё и является суффиксом паттерна), но её длина больше. Поэтому мы можем гарантировать, что подстрака с $j - M_j + 1$ по j индексы текста совпадёт с подстрокой с $i - M_j + 1$ по i индексы паттерна, а значит это тот самый блок, который можно гарантированного пропустить. Следовательно, $i = i - M_j$ и $j = j - M_j$.



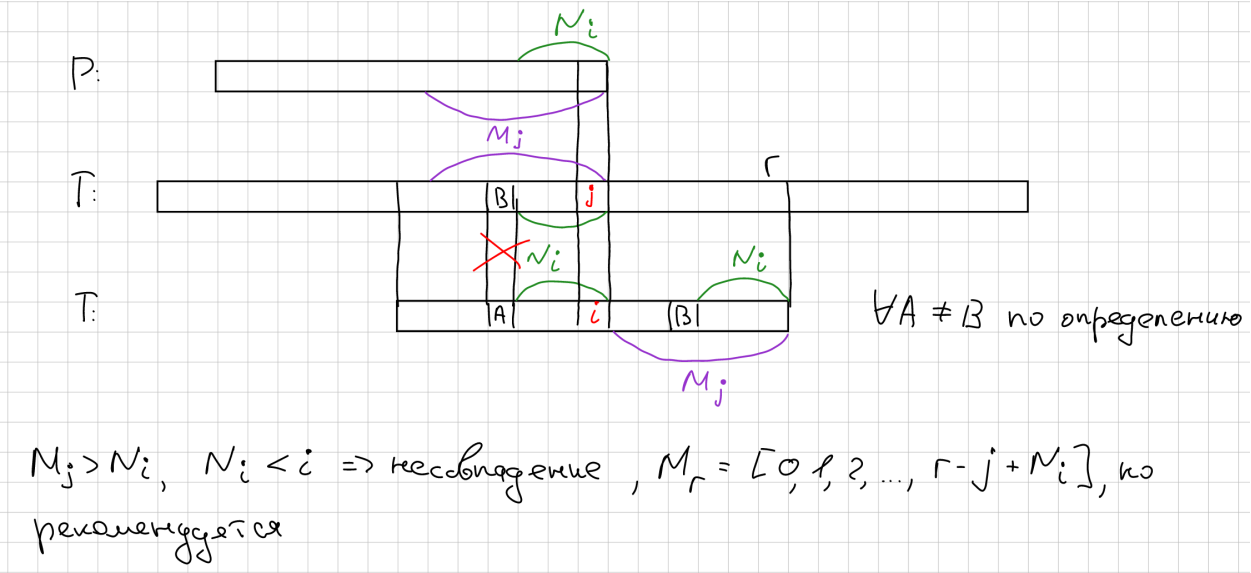
- $M_j = N_i$. Если $N_i < i$, то найдена подстрока, которая является суффиксом паттерна длиной N_i. Значит можно пропустить этот блок: $i = i - M_j$ и $j = j - M_j$.



Если $N_i = i$, то строка с 0 по i позицию является суффиком, а значит имеется имеется блок с $j - M_j + 1$ по j позиции, который уже покрывает текст. Покроем осташуюся часть строки блоком с j по r позиции: $M_r = r - j$.



- $M_j > N_i$. Если $N_i < i$, то при поптыке к j позиции приложить правый конец текста совпадёт M_j символов.. При этом существует подстрока N_i, которая является суффиксом паттерна, при этом символ перед ним совпадает с $j - N_i + 1$ символом текст. Но по определению, сейчас прикладывается не суффикс паттерна, а его ещё одно вхождение, символ перед которым отличается от того, что стоит перед суффиксом. Следовательно, символы $i - N_i$ и $j - N_i$ отличаются. Значит это гарантированное несовпадание, а $M_r = r - j$.



НО, если $N_i = i$, то существует префикс в паттерне, который является его суффиксом, и это значит, что найдено вхождение (аналогично предыдущему пункту).

Ахо Корасик

Алгоритм был придуман студенткой Маргрет Корасик под руководством Альфреда Ахо. Заключается он в том, чтобы преобработать множество паттернов таким образом, что в случае, если один паттерн является подстрокой другого паттерна, то будут созданы ссылки для перехода в него. Таким образом при поиске в тексте большого количества паттернов потребуется всего один проход по всей длине текста.

Препроцессинг

Связи неудач - это ссылка из вершины trie в другую вершину, путь которой от корня является подстрокой изначального паттерна.
Связи выхода - это ссылка из вершины trie в другую вершину, которая говорит, что в этом месте заканчивается какой-либо паттерн, следовательно, найдено его вхождение в текст

Алгоритм:

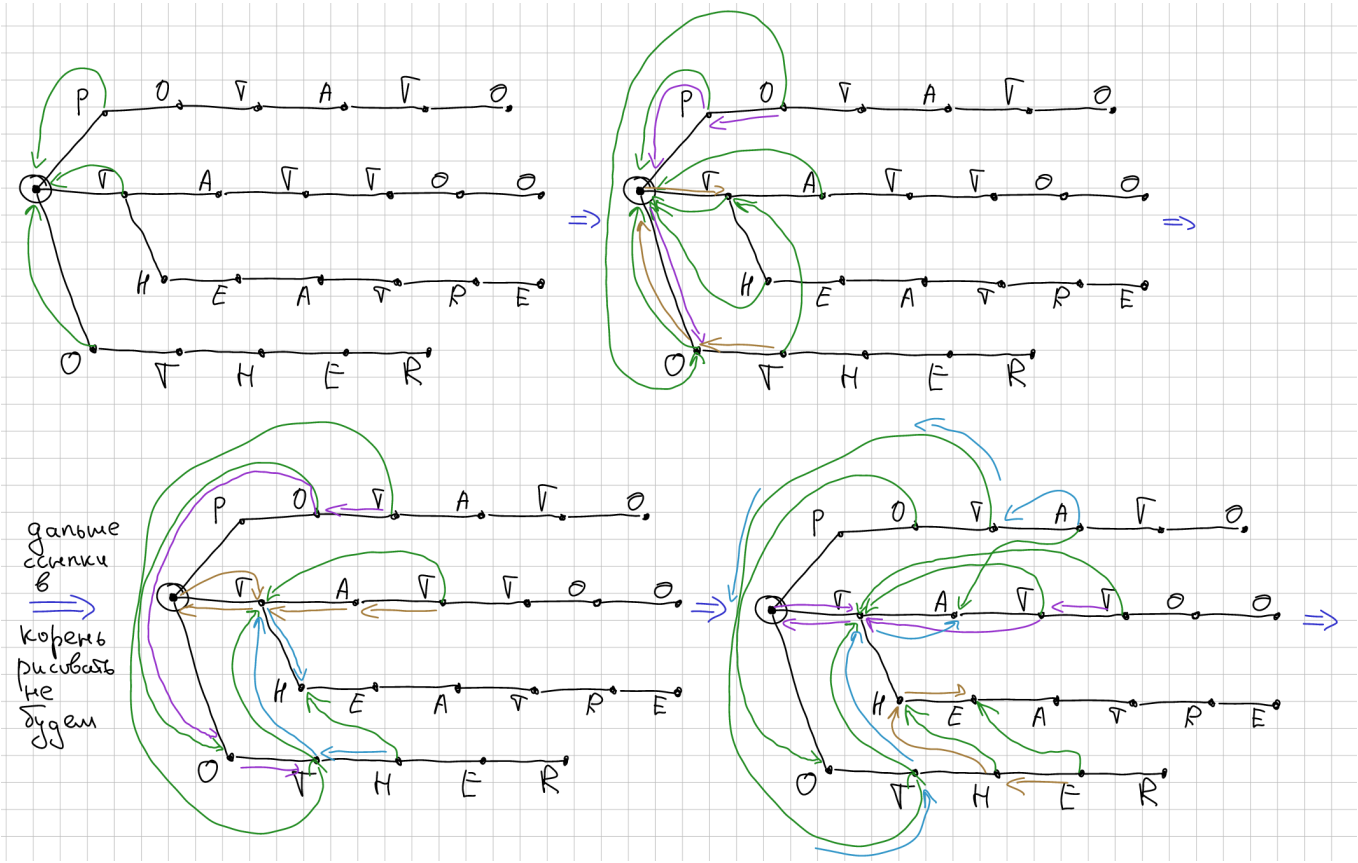
1. Построить trie для всех паттернов.
2. Для всех подстрок паттернов длины 1 ставим связи неудач в корень.
3. Для всех подстрок длины больше 1 действуем по следующему алгоритму:
    1. Запоминаем значение символа x узла X, в котором находимся. Откатываемся на один узел назад. Идём по имеющейся связи неудач в узел Y. Среди детей узла Y ищем x.
    2. Если $x \in Y$, то переходим по ссылке в узел Y и ставим связи неудач из узла X в узел Y. Делаем дополнительную проверку, не является ли узел Y листом. Если является, то какой-то паттерн является подстрокой другого паттерна, строим

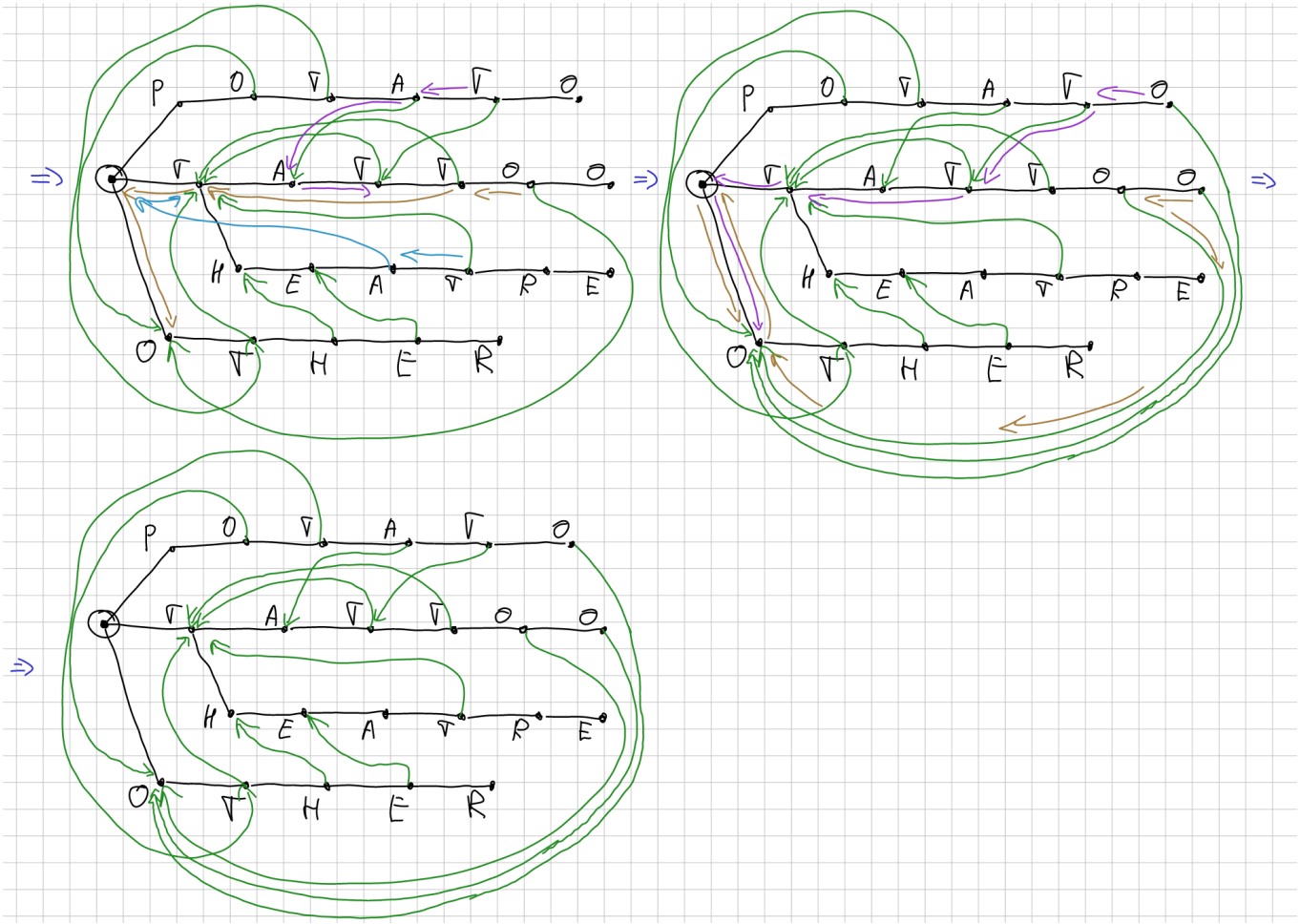
связь выхода. Если из узла Y есть связь неудач, то продолжаем идти по ней согласно п.2. и п.3. (x и X остаются теми же).

3. Если $x \notin Y$, то переходим по связи неудач из узла Y в узел Y'. Делаем повторную проверку аналогично п.2. и п.3. (x и X остаются теми же).

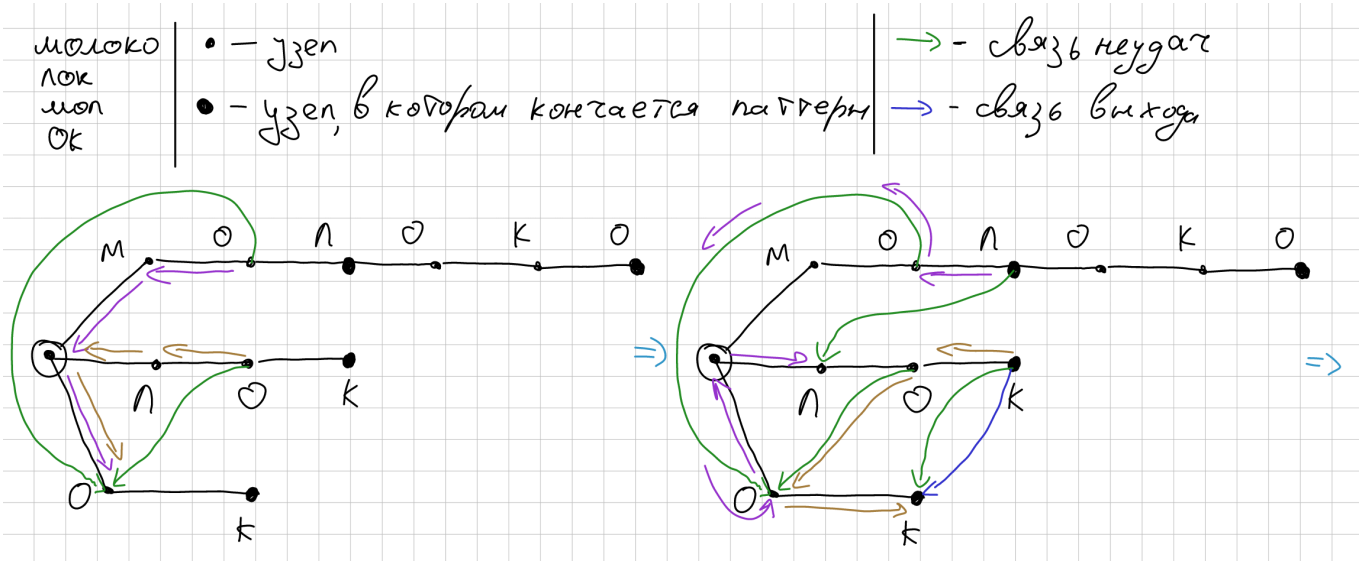
Сложность препроцессинга: $O(m + k)$, где $m = \sum_i |P_i|$ и k - количество вхождение паттернов в другие. Т.е. построение trie, настройка связей неудач и связей выхода выполняются за линейной время.

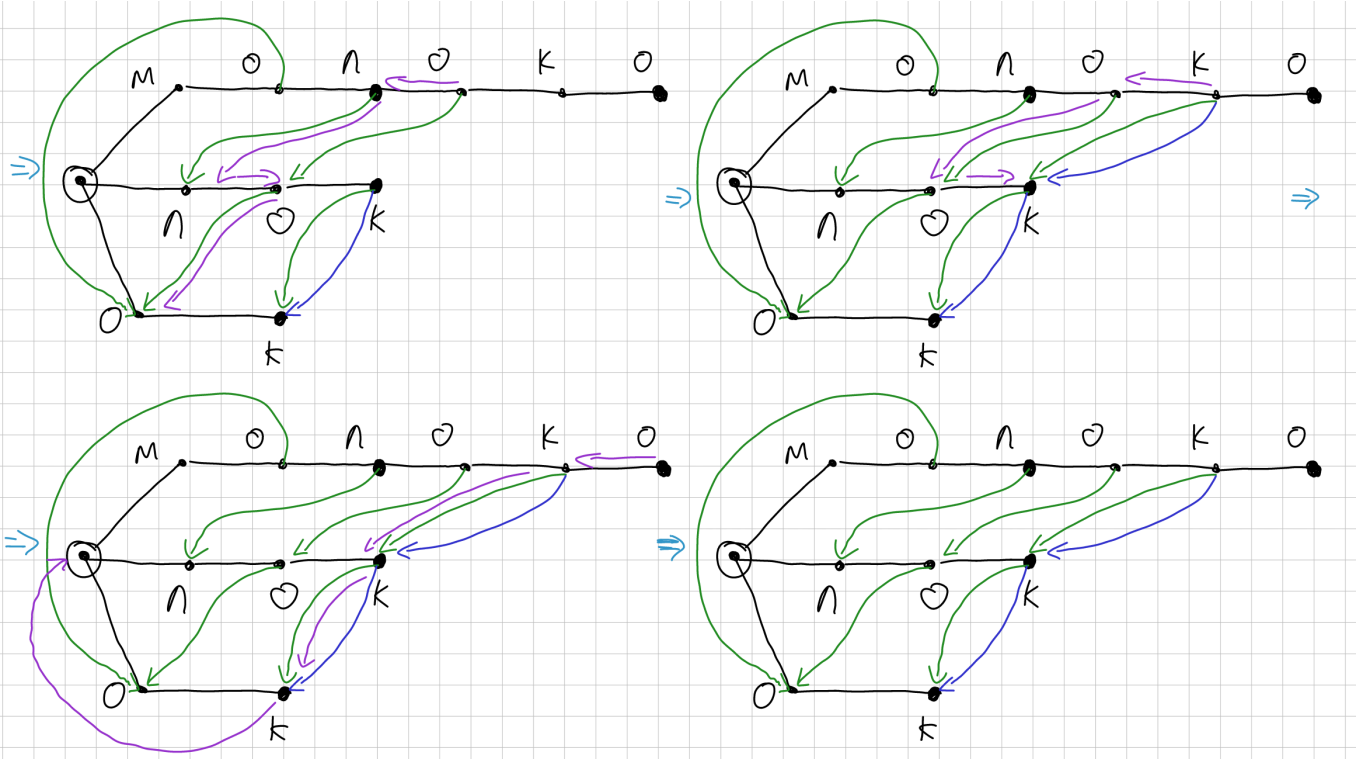
Пример из Гасфилда, но поэтапный:





Пример, когда паттерн является подстрокой другого паттерна:

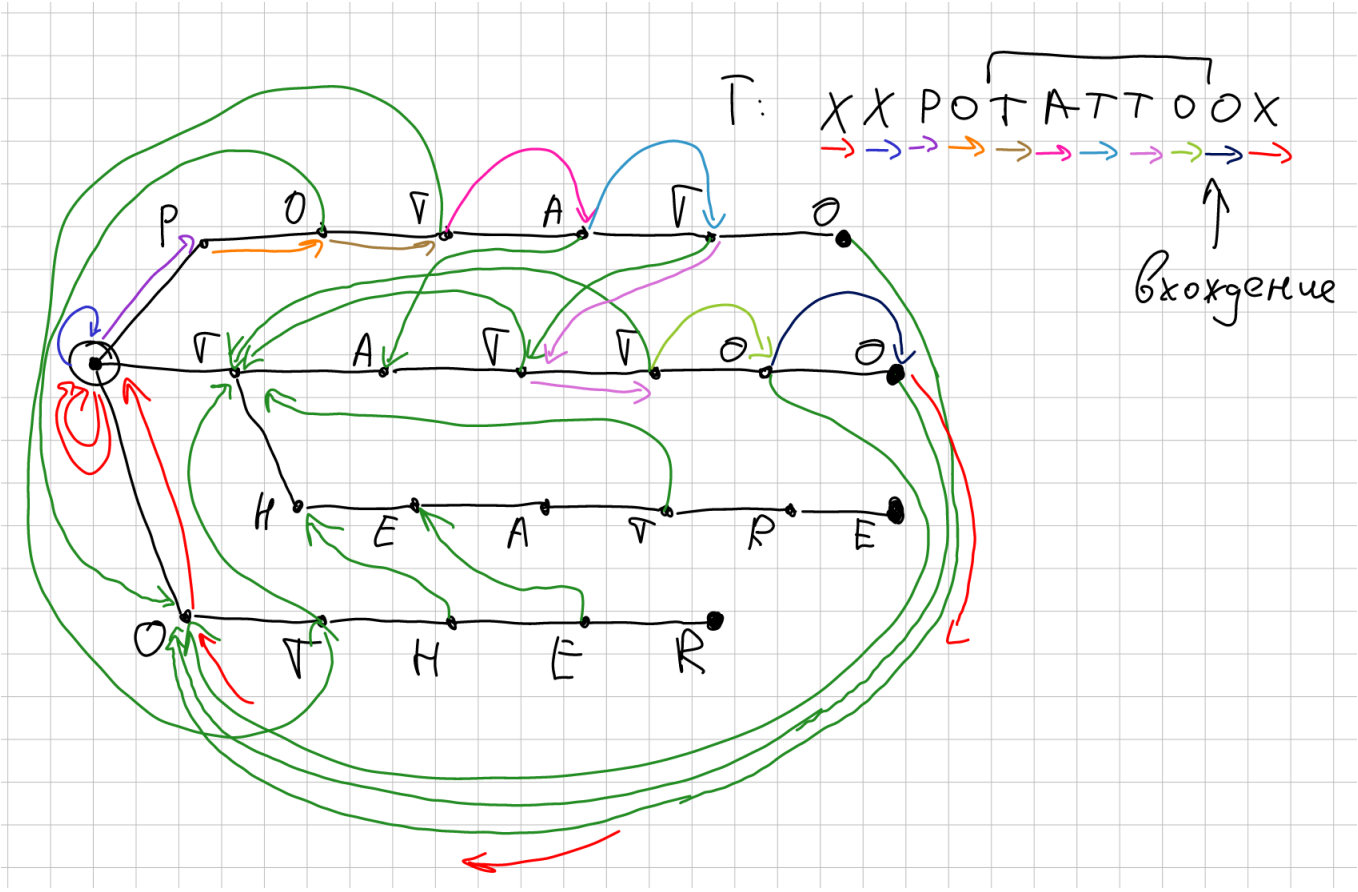




Алгоритм

1. Читаем текст слева направо посимвольно.
2. Начинаем сравнивать символ текста со всеми детьми в корне префиксного дерева.
    1. Если такой рёбнок имется, переходим по его ссылке. После перехода проверяем, есть ли связи выхода из этого узла. Если есть, то найдено вхождение одного из паттернов в текст.
    2. Если нет, то считываем следующий символ текста и повторяем п.2., иначе переходим к п.3.
3. Считываем следующий символ текста. Ищем среди детей текущего узла в префиксном дереве.
    1. Если он имеется среди детей текущего узла, то переходим по его ссылке.
    2. Иначе переходим по связи неудач и заново выполняем проверку п.3.1.
    3. Если в результате перехода по ссылке неудач оказались в корне, то переходим к п.2.

Пример из Гасфилда:



Пример, когда паттерн является подстрокой другого паттерна:

